SPDZCoder Reproduction and Privacy Modification

Girish Nallan Chakravathy
Prepared for Professor Prabhat Kumar, LUT University
22 October 2026 | Draft for author review

Scope and main finding

I reproduced the evaluation of authentic saved SPDZCoder programs using the authors' original MASCOT evaluator and implemented a limited source modification to keep intermediate comparison values secret. Across 626 original-versus-modified candidate pairs, two programs changed and both retained native acceptance. Targeted private-input tests reduced observed explicit intermediate-opening events from 50 to zero while preserving tested authorized outputs. These results support the specific disclosure repair; they do not establish an overall accuracy gain or universal confidentiality.

The historical GPT-4/Azure generation sequence and the genuine without-refactoring ablation remain **NOT REPRODUCED**. This report distinguishes published claims, archived author decisions, fresh native execution of saved candidates, and new experiments on the modified code.

Methodology of SPDZCoder

SPDZCoder addresses the gap between ordinary Python and MP-SPDZ's restricted, data-oblivious programming model. A secret comparison cannot normally control a public Python branch; arithmetic selection and supported control structures must preserve secrecy. The paper combines expert-derived rules with language-model translation [1].

Its three stages are refactoring Python into Canonical Form Python using ten refactoring rules; generating MP-SPDZ with selected demonstrations and self-reflection; and repairing code using compiler, runtime or functional-test feedback. The reported configuration is GPT-4-turbo-2024-04-09, temperature 0.7, two candidates per task and up to three repair rounds. SPDZEval contains 313 tasks. The release separates NumPy operations and universal functions into two directories, producing seven physical categories corresponding to the paper's six groups [1,2].

I mapped these stages to the released source and inspected seven authentic saved-stage examples. Released refactoring and repair control flow differs from the idealized method, and saved snapshots omit complete model conversations. Consequently, the source audit and saved-state traces do not establish regeneration of the historical pipeline [3].

Reproduction protocol

The baseline uses the unchanged published author archive, original candidates, templates and acceptance rule. I pinned MP-SPDZ 0.3.6 at `6cc3fccef0904764364eb8e285519cd34028f325`, the original evaluator and runtime hashes, and a Python 3.11.9 container. Both MASCOT parties ran locally within a non-root, read-only, network-disabled container with two CPUs and 6 GiB RAM. These are disclosed reproduction-environment choices, not recovered historical hardware specifications [3].

Original task identities, source insertion, inputs, attempts, stdout/stderr and party logs are preserved. Saved acceptance flags are comparison metadata; fresh decisions come from executed programs. No historical model calls were recreated. Full artifact pins and rerun commands accompany the report in `docs/final_report_evidence.md` [7].

Reproduced evaluation results

Table 1 uses the original evaluator's candidate acceptance decisions. Each row covers 313 tasks with two saved candidates per task. “Either” counts tasks with at least one accepted candidate. The first-candidate and either-candidate percentages reproduce the authors' recorded metric convention; neither is a proof of general program correctness.

Evidence and system	First	Second	Either
Archived author flags for final SPDZCoder	269	265	288
Archived author flags for stage 2 without repair	229	213	261
Archived author flags for GPT-4 API-doc baseline	95	114	156
Fresh original MASCOT final SPDZCoder	268	265	288
Fresh original MASCOT stage 2	229	216	262
Fresh original MASCOT GPT-4 API-doc baseline	95	112	154

Table 1. Archived decisions and independently re-executed original candidates. Final-stage run 38703569637, attempt 2; stage-2 source run 39075744038 and artifact-only recovery 39080161119 [3,4].

The archived final SPDZCoder counts give 269/313 = 85.94% and 288/313 = 92.01%, matching the paper's reported headline results. Fresh final-stage replay gives 268/313 = 85.62% and 288/313 = 92.01%. This agreement in the union count does not reproduce historical generation or erase the differing individual decisions.

Final-stage replay includes 1,252 candidate records across SPDZCoder and the GPT-4 API-documentation baseline, with 1,280 attempts. SPDZCoder has 75 compilation failures and one final timeout. The baseline has 312 compilation failures, 70 runtime failures and two final timeouts. Five archived-versus-fresh decision disagreements are retained rather than filtered [3].

Stage 2 contains 626 candidates and 644 attempts: 147 compilation failures, 479 completed runtimes, 445 acceptances and 34 completed-runtime rejections. Fourteen candidates retried; 18 timed-out attempts remain in the logs. Three saved-to-fresh disagreements account for the changed stage-2 totals. The archived stage-2 comparison recovers the available without-repair result; it is not the missing without-refactoring ablation [4].

Meaning and limits of the original oracle

For a non-timed-out attempt, the released evaluator accepts a case-insensitive `pass` substring in stdout unless stderr contains `Traceback`; it does not require a successful process exit. Diagnostics demonstrated false acceptance from a filename and from signed-relative numerical checks. One native template lacks a functional assertion. I retained these rules for comparability and report “original-oracle acceptance” rather than treating every accepted program as correct [3,5].

The separate privacy diagnostics use independently computed authorized outputs and explicit opening traces. Their results are excluded from Table 1 and from native benchmark rates. Earlier independently reconstructed fixtures and cleartext-emulator studies are also excluded from the original-system results in this report.

Limitation and source modification

A functionally accepted translation can reveal information beyond its authorized output. My diagnostic policy makes array length and type public, keeps party-0 array values private, and authorizes only the final count or Boolean result. The observer represents another protocol participant able to see explicit openings. It prints values already opened by the candidate and introduces no additional reveal operation [5].

For authentic `count_distinct` candidate 1, private arrays `[1,1,2]` and `[1,2,2]` both return 2, but the original comparison-opening traces are `[0,1,1]` and `[1,1,0]`. Thus, the same permitted result can accompany distinguishable information about the input. This is a new private-input diagnostic of an original translation, not evidence of a historical breach of the public benchmark.

I added `preserve_secret_arithmetic` to both original entry points, `src/pipeline.py` and `src/pipeline-mp.py`, after optional feedback and before output serialization. The original-source patch is `research/modifications/original-pipeline.patch`; the implementation is `research/modifications/secret_arithmetic.py`.

```
from secret_arithmetic import preserve_secret_arithmetic
output, _privacy_rewrites = preserve_secret_arithmetic(output, test_function_name)
```

The following expressions are from the authentic saved final-stage records in `data/stage_3_output/experiment-1-loop_test.jsonl` and their preserved transformed outputs [5].

```
# count_distinct candidate 1: original, then modified
unique *= (arr[j] != arr[i]).reveal()
unique *= arr[j] != arr[i]

# is_ascending candidate 0: original, then modified
result = result * (1 - condition.reveal())
result = result * (1 - condition)
```

These paired lines show alternatives, not statements to execute consecutively. The finalizer parses the AST, identifies supported comparison openings multiplied into a secret `sint` accumulator, and verifies that all relevant bindings preserve that invariant. Only those selected calls are replaced. The replacement visitor is:

```
class Replace(ast.NodeTransformer):
    def visit_Call(self, node):
        if node in replacements:
            return ast.copy_location(replacements[node], node)
        return self.generic_visit(node)
```

Keeping a valid comparison bit, or its complement, secret preserves the numeric product under the stated MP-SPDZ scalar-type and range assumptions. Unsupported syntax, rebinding, zero folding and incompatible expressions are rejected by the transformation; unchanged fallback preserves the original source bytes. This is a conditional rationale and finite-test result, not a general equivalence or noninterference proof. The mixed opened-operand `lower_bound` witness remains outside the patch's scope [5,6].

The exact added finalizer was exercised on saved candidates. The preceding model-dependent stages of the patched pipeline were not executed, so this does not establish improved generation quality.

Controlled comparison and measured costs

The full paired experiment applies the finalizer to all 626 authentic final-stage candidates while preserving native templates, inputs, evaluator and pinned execution conditions. Pair order alternates. The run contains 1,252 execution records and 1,280 attempts [6].

Measure	Original	Modified
First candidate accepted out of 313	270	271
Second candidate accepted out of 313	266	263
Either candidate accepted out of 313	290	290
Compilation failures out of 626	75	75
Candidate sources changed by finalizer	0	2
Changed candidates retaining acceptance	2 accepted	2 accepted

Table 2. Full paired run 38989067493. These measurements are from a separate execution from Table 1; their baseline counts must not be mixed.

Only `count_distinct` candidate 1 and `is_ascending` candidate 0 change. Both remain accepted. All four acceptance flips occur among the 624 source-identical controls, with identical compiled-input bytes and compiler fingerprints. In a 16-execution original-only follow-up, `next_perfect_square` candidate 1 accepted in three of four trials. The cause of variability remains unproven. The aggregate changes therefore cannot be credited to the patch as an accuracy gain or regression [6].

Targeted testing uses 32 pairs: two native pairs, 24 observed private-input pairs, four unobserved private-control pairs and two synthetic fallback pairs. All 64 executions compile and complete. Tested authorized outputs are preserved, while the 24 observed private-input pairs show **50 original explicit intermediate-opening events and zero modified events**. These are events, not 50 vulnerabilities. Internal cryptographic protocol openings are a different quantity [5].

Native candidate and variant	Runtime s	Global MB	Integer triples	Compiler opens
count_distinct original	18.6831	390.364	2,917	191
count_distinct modified	20.6538	429.331	3,010	98
is_ascending original	8.822	195.163	2,042	35
is_ascending modified	8.86707	195.164	2,059	18

Table 3. Single-run native costs from full paired run 38989067493, including preprocessing. These timings are not the separate targeted-smoke timings [6].

The repair increases secret multiplication costs and, especially for `count_distinct`, communication and observed runtime. Compiler open counts include protocol operations and cannot be equated with explicit candidate-opening events. These isolated measurements do not establish a performance distribution or a speed advantage.

Reproduction difficulties and unresolved experiments

The author release became available through the supplied Zenodo link. It has a `requirement.txt` versus README `requirements.txt` mismatch and lacks complete historical environment/model pins. The original scripts target MP-SPDZ 0.3.6. Local namespace restrictions led to the confined CI Docker execution route. All selected environment choices and dependency hashes are recorded [2,3].

The first full final-stage attempt suffered a truncated download; failed jobs were retried and the incomplete dataset was preserved. Stage-2 run **39075744038 remains FAILED** because aggregation used the wrong artifact-root prefix despite seven successful native jobs. Separate artifact-only recovery **39080161119 succeeded without rerunning candidates**. Compiler failures, timeouts, the 70 baseline runtime failures, oracle defects and unclassified stderr remain in the evidence [3,4,7].

Released stage-1 and repair control flow does not fully explain saved outputs. Intermediate model messages are missing, and 50 branch candidates change between stages 2 and 3 despite zero recorded repair tokens. I did not invent missing conversations or treat saved token counts as complete billed usage [3].

NOT REPRODUCED: the historical GPT-4/Azure generation sequence, because the exact deployed revision and full call history are unverified; and the genuine SPDZCoder without-refactoring ablation, because no attributable original candidate corpus was recovered. Audits covered 498 relevant source files, 35 saved evaluation groups, embedded Git history and release lineage. All 313 original input/instruction pairs were recovered, with six direct Python-parse exceptions preserved. Matching UniTrans baseline counts do not identify the missing SPDZCoder experiment. Additional author artifacts could resolve these gaps; a newly generated run would be a new experiment [7].

Reproducibility and references

The evidence snapshot is repository commit `c237aa4a6d98a16b141a9b178a046a121ea1e659`. It passed 189 regression tests, compiler/emulator checks, provenance, difficulty accounting and a fresh 31-artifact audit. Verification uses separate audit procedures and AI-assisted reviews, not external certification. Detailed source paths, hashes, commands and raw-artifact locations are in the companion evidence guide. The measured contribution is removal of the supported intermediate-opening pattern, with preserved tested outputs and disclosed extra costs.

[1] X. Dong, P. Xin, J. Li and W. Xu. *SPDZCoder: Combining Expert Knowledge with LLMs for Generating Privacy-Computing Code*. arXiv:2501.00363v2, 21 March 2025.

[2] X. Dong. *SPDZCoder Artifacts*, v2. Zenodo 18152253, 2026.

[3] Original-source audit and final-stage run 38703569637.

[4] Stage-2 acceptance receipt and recovery 39080161119.

[5] Private-input diagnostics and targeted comparison.

[6] Full paired evidence and costs, run 38989067493, and original-only follow-up 39043579023.

[7] Difficulty register, merged-main raw-evidence audit 39541724665 and companion evidence guide.